Code:

```
import heapq
from collections import defaultdict

def a_star_search(graph, heuristics, s, G):
    open_list = []
    closed = set()
    parent = {s: None}
    g_cost = {s: 0}

    # Push (f_cost, h_cost, node) into open_list: f(s) = g(s) + h(s)
    heapq.heappush(open_list, (heuristics[s], heuristics[s], s))

    while len(open_list) != 0:
        f, h, n = heapq.heappop(open_list)

        if n == G:
            curr = G
            path = []
            while curr is not None:
                path.append(curr)
                curr = parent[curr]

            print("Path:", path[::-1])
            print("Total Cost:", g_cost[G])
            return path[::-1]

        if n in closed:
            continue

        closed.add(n)

        for successor, weight in graph[n]:
            tentative_g = g_cost[n] + weight

            if successor not in g_cost or tentative_g < g_cost[successor]:
                g_cost[successor] = tentative_g
                f_cost = tentative_g + heuristics[successor]
                parent[successor] = n
                heapq.heappush(open_list, (f_cost, heuristics[successor], successor))
```

```
            if successor in closed:
                closed.remove(successor)

    print("G node is not reachable from S node")
    return None

def main():
    n = int(input("No of vertices: "))
    heuristics = {}
    print("Enter heuristic values (node heuristic):")
    for _ in range(n):
        node, h = input().split()
        heuristics[node] = float(h) if '.' in h else int(h)

    e = int(input("No of edges: "))
    graph = defaultdict(list)
    print("Enter edges with weight (u v weight):")
    for _ in range(e):
        u, v, w = input().split()
        weight = float(w) if '.' in w else int(w)
        graph[u].append((v, weight))
        graph[v].append((u, weight))

    s = input("s node: ").strip()
    G = input("G node: ").strip()

    a_star_search(graph, heuristics, s, G)

if __name__ == "__main__":
    main()
```

Output:

```
(base)  sanjaywaradkar@Sanjays-MacBook-Air-2 AI_Lab % /opt/homebrew/bin/python3
"/Users/sanjaywaradkar/Desktop/SEM V/AI_Lab/exp8/a_star.py"
No of vertices: 6
Enter heuristic values (node heuristic):
S 8
A 6
B 5
C 4
D 3
G 0
No of edges: 7
Enter edges with weight (u v weight):
S A 2
S B 4
A C 3
A D 3
B D 2
C G 5
D G 4
s node: S
G node: G
Path: ['S', 'A', 'D', 'G']
Total Cost: 9
```